

Два варианта темы ВКР

Темы, содержание работы с описанием разделов и ожидаемый результат по экранам и компонентам для двух тематик из списка кафедры.

ВАРИАНТ 1 · ТЕМАТИКА 37

Разработка дизайна интерфейса BI-системы самообслуживания для пользователей без аналитической подготовки (на примере системы «Срез» для сетей общепита)

ВАРИАНТ 2 · ТЕМАТИКА 48

Способы отображения серверной логики в интерфейсе веб-приложения при фуллстек-разработке (на примере CRM-системы «Верстак» для сервисных центров)

Как читать

- Оба проекта вымышленные и не связаны с существующими системами. Названия продуктов «Срез» и «Верстак» рабочие: перед утверждением проверить, что они не заняты действующими продуктами.
- Научные источники названы по памяти. Перед включением в список литературы сверить выходные данные по первоисточникам.
- Окончательную формулировку темы согласовать с руководителем: кафедры часто правят слова «разработка», «проектирование», «особенности» под свой шаблон.
- Объём и оформление работы сверить с методичкой кафедры.

Состав документа

Вариант 1. BI-система «Срез»: тема, содержание, результат.

Вариант 2. CRM-система «Верстак»: тема, содержание, результат.

Приложение А. Что такое BI-система самообслуживания.

Приложение Б. Кто такой пользователь без аналитической подготовки.

Приложение В. Как отображается серверная логика в интерфейсе: примеры.

BI-система «Срез»

ТЕМА ВКР

Разработка дизайна интерфейса BI-системы самообслуживания для пользователей без аналитической подготовки (на примере системы «Срез» для сетей общепита)

- **Продукт:** «Срез», веб-сервис аналитики для сетей кофеен и небольших ресторанов на 3-30 точек.
- **Отрасль:** общепит, малый и средний бизнес.
- **Пользователи:** владелец сети, управляющий точкой, администратор аккаунта. Аналитиков среди них нет.
- **Данные:** выгрузки кассовых систем, Excel-таблицы, отчёты сервисов доставки.
- **Гипотеза:** если интерфейс скрывает термины модели данных и предупреждает о типовых ошибках, пользователи без подготовки решают аналитические задачи в «Срезе» верно чаще, чем в DataLens.

Содержание

Введение

Актуальность (уход зарубежных BI-систем, малый бизнес без аналитиков), объект, предмет, цель, задачи, гипотеза и методы исследования.

Глава 1. Теоретические основы проектирования интерфейсов BI-систем

- 1.1. **Понятие, назначение и архитектура BI-систем.** Путь данных от источника до дашборда и роль каждого модуля глазами пользователя.
- 1.2. **Концепция самообслуживания в бизнес-аналитике.** Чем самообслуживание отличается от классической BI и на каких шагах оно ломается у непрофессионалов.
- 1.3. **Закономерности зрительного восприятия и визуализации данных.** Иерархия визуальных кодировок, преаттентивные признаки и правила выбора графика под задачу.
- 1.4. **Когнитивная нагрузка и постепенное раскрытие сложности.** Как дозировать сложность модели данных для человека без подготовки.
- 1.5. **Нормативные требования к эргономике и доступности.** Положения ГОСТ Р ИСО 9241-110 и WCAG 2.2 применительно к интерфейсам с графиками.

Выводы по главе 1. Теоретические принципы, на которые опирается проект.

Глава 2. Анализ интерфейсов BI-систем и потребностей пользователей

- 2.1. **Рынок BI-систем в России после 2022 года.** Какие системы доступны и на какой уровень подготовки пользователя они рассчитаны.
- 2.2. **Сравнительный анализ интерфейсов BI-систем.** DataLens, Visiology, Metabase и Power BI как мировой эталон по критериям порога входа и защиты от ошибок.
- 2.3. **Исследование пользователей общепита.** Интервью с владельцами и управляющими кофеен: какие вопросы они задают данным и как сейчас получают ответы.
- 2.4. **Персоны и сквозные сценарии.** Персоны владельца сети и управляющего точкой и их путь от вопроса до ответа.
- 2.5. **Требования к интерфейсу «Среза».** Перечень требований, выведенный из теории, анализа аналогов и интервью.

Выводы по главе 2. Требования, которые переходят в проектирование.

Глава 3. Разработка дизайна интерфейса BI-системы «Срез»

- 3.1. **Концепция и информационная архитектура.** Позиционирование продукта, карта разделов и навигация.
- 3.2. **Сквозной сценарий.** Путь от выгрузки из кассовой системы до дашборда и еженедельной рассылки.
- 3.3. **Дизайн модулей подключения и подготовки данных.** Загрузка, распознавание таблиц, связи и вычисляемые поля с защитой от типовых ошибок.
- 3.4. **Дизайн конструктора визуализаций.** Построение графика без аналитических терминов, подбор типа графика и предупреждения об ошибках.
- 3.5. **Дизайн дашбордов, публикации и совместной работы.** Сборка и просмотр дашбордов, доступ по точкам, комментарии, рассылки и оповещения.
- 3.6. **Дизайн-система и UI-кит.** Токены, компоненты, библиотека графиков и цветовая система для данных в светлой и тёмной темах.
- 3.7. **Интерактивный прототип.** Прототип сквозного сценария в Figma и прототип конструктора в коде для теста перетаскивания полей.
- 3.8. **Юзабилити-тестирование и оценка результатов.** Сравнение «Среза» с DataLens на одинаковых задачах и проверка гипотезы.

Выводы по главе 3. Что спроектировано и что показал тест.

Заключение

Итоги работы, результат проверки гипотезы, ограничения исследования и направления развития.

Список использованных источников

Научные работы, стандарты и документация BI-систем.

Приложения

Сценарий интервью, анкеты, протоколы тестирования и полноразмерные макеты.

Результат

Итого: 20 экранов для десктопа, 6 адаптивных макетов, около 25 кадров состояний, UI-кит, библиотека графиков и два прототипа.

Экраны

Старт

1. **Первый вход.** Выбор типа заведения и вопроса («выручка по точкам», «средний чек», «популярные позиции»), кнопки «Подключить данные» и «Открыть демо». Отражает: вход без пустого экрана и постепенное раскрытие сложности.
2. **Главная.** Последние дашборды, шаблоны, автоматические находки («выручка на Ленинском упала на 18% к прошлой неделе»), свежесть данных. Отражает: система сама подсказывает, куда смотреть.

Данные

3. **Источники данных.** Подключённые кассы и файлы со статусом синхронизации, временем обновления и ошибками. Отражает: пользователь всегда знает, насколько свежи цифры.
4. **Мастер подключения.** Шаги: тип источника, файл или доступ, предпросмотр, сохранение. Отражает: сложная операция разбита на короткие шаги.
5. **Распознавание таблицы.** Система определила типы колонок (дата, деньги, текст) и простым языком подсветила проблемы: пустые ячейки, дубли, разные форматы дат. Отражает: проблемы качества данных видны до построения графиков.
6. **Связи таблиц.** Визуальная схема связей чеков, точек и меню, предупреждение о задвоении строк с примером на цифрах. Отражает: интерфейс ловит самую опасную ошибку непрофессионала раньше, чем она исказит выручку.
7. **Вычисляемое поле.** Редактор формулы в стиле Excel с подсказками и живым результатом («Средний чек = Выручка / Число чеков»). Отражает: формулы без SQL.

Конструктор

8. **Конструктор визуализации.** Поля с понятными названиями («Что считаем» и «В разрезе чего» вместо мер и измерений), холст, рекомендованный тип графика, выбор агрегации с подсказкой. Отражает: центральный экран гипотезы.
9. **Фильтры и сравнение периодов.** Фильтры по периоду, точке и категории меню, режим «этот месяц против прошлого». Отражает: частый вопрос владельца решается в два клика.
10. **Детализация.** Переход от столбца графика к строкам, из которых сложилась цифра. Отражает: цифре можно доверять, потому что её можно проверить.

Дашборды

11. **Каталог шаблонов.** Готовые дашборды для общепита: выручка, средний чек, загрузка по часам, ABC-анализ меню. Отражает: старт с готового решения.
12. **Редактор дашборда.** Сетка, добавление графиков, текстовые пояснения, общий фильтр на весь дашборд. Отражает: композицию и иерархию информации.
13. **Дашборд владельца сети.** Плитки ключевых показателей, динамика выручки, сравнение точек, тепловая карта загрузки по часам и дням недели. Отражает: закономерности восприятия и переход от общего к частному.
14. **Дашборд управляющего точкой.** Только своя точка, план и факт, текущая смена. Отражает: разный уровень детализации для разных ролей на одних данных.

Совместная работа

15. **Публикация и доступ.** Кому показывать дашборд и какие точки видит каждый человек. Отражает: ограничение доступа до уровня строк без технических терминов.
16. **Комментарии к данным.** Обсуждение, привязанное к точке на графике («что случилось 14 августа?»). Отражает: командную работу с данными.

17. **Рассылки.** Отчёт в почту или мессенджер по расписанию, с предпросмотром письма.

Отражает: данные приходят к человеку сами.

18. **Оповещения по порогам.** Правило собирается как фраза: «Сообщить, если выручка точки упадёт больше чем на 20% к прошлой неделе». Отражает: настройку условия без формул.

Администрирование

19. **Пользователи и роли.** Сотрудники, роли, приглашения.

20. **Центр уведомлений.** Сработавшие оповещения, упоминания в комментариях, ошибки синхронизации.

Адаптивные макеты (6): дашборд владельца, дашборд управляющего и оповещение с переходом к графику, каждый на ширине планшета и телефона.

Окна, компоненты, состояния

- **Модальные окна (5):**
 - выбор агрегации с объяснением «сумма или среднее»;
 - предупреждение о задвоении при связи таблиц;
 - настройка графика;
 - публикация;
 - удаление дашборда с перечнем мест, где он используется.
- **Предупреждения конструктора (4 типа):** неверная агрегация, неподходящий тип графика, задвоение строк, слишком мало данных для вывода.
- **Подсказки терминов:** объяснение слова по наведению («медиана», «доля») на примерах из общепита.
- **Состояния (около 25 кадров):** пустой дашборд, загрузка, данные устарели, ошибка источника, нет данных за период, первый запуск конструктора.
- **Библиотека графиков (9 типов):** плитка показателя, линия, вертикальные столбцы, горизонтальные столбцы, план и факт, тепловая карта, таблица, сводная таблица, кольцевая диаграмма с ограничением до 5 долей.
- **Цветовая система данных:** категориальная, последовательная и расходящаяся палитры для светлой и тёмной тем, проверенные на нарушения цветового зрения.
- **UI-кит:** кнопки, поля, выпадающие списки, чипы полей данных, фильтры, панели, уведомления.

Прототипы и исследование

- **Прототип в Figma:** сквозной сценарий по экранам 1-18.
- **Прототип конструктора в коде (экран 8):** перетаскивание полей работает по-настоящему, тест проходит на нём.
- **Тест:** 12-20 участников уровней 1-2 (см. приложение Б), перед началом Mini-VLAT.
 - Четыре задачи, каждая в «Срезе» и в DataLens: найти точку с падением выручки, посчитать средний чек, связать две таблицы, собрать дашборд и настроить рассылку.
 - Метрики: успешность, время, ошибки, SUS.

CRM-система «Верстак»

ТЕМА ВКР

Способы отображения серверной логики в интерфейсе веб-приложения при фуллстек-разработке (на примере CRM-системы «Верстак» для сервисных центров)

- **Продукт:** «Верстак», веб-система учёта заказов для сервисных центров по ремонту телефонов, ноутбуков и бытовой техники.
- **Отрасль:** сервисные центры и их сети, малый и средний бизнес.
- **Пользователи:** приёмщик, мастер и руководитель. Клиент видит публичную страницу статуса ремонта.
- **Почему эта отрасль:** в ней есть все шесть аспектов серверной логики:
 - многолетняя история заказов;
 - фоновые SMS-рассылки;
 - восемь статусов заказа;
 - два мастера претендуют на последнюю запчасть;
 - мастеру нельзя видеть закупочные цены;
 - на приёмке один компьютер делят несколько сотрудников.
- **Гипотеза:** в интерфейсе, спроектированном по карте соответствия, пользователи верно понимают серверные ситуации и выбирают правильное следующее действие чаще, чем в наивном интерфейсе.

Содержание

Введение

Актуальность (рост сложности веб-приложений и разрыв между макетом и серверной реализацией), объект, предмет, цель, задачи, гипотеза и методы исследования.

Глава 1. Теоретические основы проектирования интерфейса в фуллстек-разработке

- 1.1. **Фуллстек-разработка в процессе проектирования интерфейса.** Что меняется в проектировании, когда один специалист ведёт и интерфейс, и серверную часть.
- 1.2. **Клиент-серверная архитектура веб-приложения с точки зрения пользователя.** Какие серверные процессы пользователь не видит, но ощущает через задержки, отказы и расхождения данных.
- 1.3. **Разрывы между макетом и реализацией интерфейса.** Исследования того, где теряется замысел при передаче макета в разработку.
- 1.4. **Разрывы исполнения и оценки.** Модель Нормана как основа требования: пользователь должен понять, что система сделала с его действием.
- 1.5. **Состояния интерфейса и диаграммы состояний.** Модель UI Stack и диаграммы Харела как язык описания поведения экрана.

Выводы по главе 1. Теоретическая рамка метода.

Глава 2. Анализ отображения серверной логики в интерфейсах веб-приложений

- 2.1. Классификация аспектов серверной логики.** Шесть аспектов (данные, время, жизненный цикл, ошибки, доступ, сессия) с признаками и примерами.
- 2.2. Паттерны отображения в CRM-системах и дизайн-системах.** Как эти аспекты решены в CRM для сервисных центров (например, RemOnline, LiveSklad), в amoCRM и Битрикс24 и в дизайн-системах Material, Carbon, Gravity UI.
- 2.3. Анализ предметной области сервисных центров.** Процессы приёма, ремонта и выдачи техники и серверные ситуации, которые в них возникают.
- 2.4. Исследование восприятия серверных событий.** Пилотный тест: как пользователи понимают типовые сообщения о задержках, ошибках и конфликтах в существующих интерфейсах.
- 2.5. Требования к проектированию интерфейса.** Требования к отображению каждого из шести аспектов.

Выводы по главе 2. Требования, которые переходят в метод и проект.

Глава 3. Разработка дизайна интерфейса CRM-системы «Верстак»

- 3.1. Метод согласования клиентской и серверной моделей.** Пошаговый метод: сущность, её состояния, ответ сервера, экранное решение.
- 3.2. Карта соответствия серверных состояний и экранных решений.** Сводная карта по шести аспектам для всех модулей «Верстака».
- 3.3. Отображение данных и их объёма.** Список заказов, поиск и склад при базе в сотни тысяч записей.
- 3.4. Отображение времени и длительных операций.** Импорт базы, выгрузки, SMS-рассылки и события в реальном времени.
- 3.5. Отображение жизненного цикла сущностей.** Статусы заказа, история изменений и отмена ошибочного действия.
- 3.6. Отображение ошибок и расхождений данных.** Конфликт правок, гонка за последнюю запчасть и серверная проверка при вводе.
- 3.7. Отображение доступа и ограничений.** Различия интерфейса приёмщика, мастера и руководителя.
- 3.8. Отображение сессии и надёжности соединения.** Общий компьютер приёмки, обрыв сети при сохранении и технические работы.
- 3.9. Дизайн-система и рабочий прототип.** Компоненты с полным набором состояний и прототип в коде с панелью управления серверными ситуациями.
- 3.10. Апробация и оценка результатов.** Сравнение наивного и спроектированного интерфейсов на сценариях с серверными событиями.

Выводы по главе 3. Что спроектировано и что показал тест.

Заключение

Итоги работы, результат проверки гипотезы, ограничения исследования и направления развития.

Список использованных источников

Научные работы, стандарты, документация дизайн-систем.

Приложения

Полная карта соответствия, диаграммы состояний, протоколы тестов и фрагменты контракта API.

Результат

Итого: 22 экрана и около 45 кадров состояний. К ним добавляются методические артефакты: 5 диаграмм состояний, карта соответствия на 60-80 строк, каталог из 20 паттернов, дизайн-система и рабочий прототип в коде.

Экраны

Вход и рабочие места

1. **Вход и смена сотрудника.** Вход, быстрая смена пользователя на общем компьютере приёма, блокировка после неудачных попыток с таймером. Отражает: сессию.
2. **Рабочий стол приёмщика.** Очередь заказов по статусам, готовые к выдаче, индикатор соединения, новые события в реальном времени. Отражает: время и надёжность.
3. **Рабочий стол мастера.** Свои заказы, ожидающие запчасть, заказ в работе, статусы обновляются без перезагрузки. Отражает: жизненный цикл и доступ.
4. **Рабочий стол руководителя.** Заказы в работе и просроченные, нагрузка мастеров, выручка с отметкой «обновлено 2 минуты назад». Отражает: свежесть агрегированных данных.

Заказы

5. **Список заказов.** Серверные страницы, фильтры, поиск по телефону и IMEI, сортировка только по колонкам с индексом, счётчик «более 10 000». Отражает: данные и объём.
6. **Приём устройства.** IMEI проверяется на сервере во время ввода («по этому устройству уже открыт заказ №4512»), черновик переживает обрыв сети, повторное нажатие не создаёт дубль. Отражает: ошибки и надёжность.
7. **Карточка заказа.** Текущий статус и доступные переходы, «сейчас редактирует мастер Петров», запчасти, оплаты. Отражает: жизненный цикл и конфликты.
8. **История заказа.** Лента изменений «кто, что, когда» и отмена ошибочной смены статуса в течение 15 минут. Отражает: жизненный цикл.
9. **Согласование стоимости с клиентом.** SMS отправлено, доставлено, клиент открыл ссылку, согласовал или отказался. Отражает: внешние сервисы со своими статусами.
10. **Выдача и оплата.** Платёж в обработке, прошёл или отклонён, защита от двойной оплаты при повторном нажатии. Отражает: длительную операцию с деньгами.

Клиенты

11. **База клиентов.** Поиск и объединение дублей: сервер нашёл клиента с тем же телефоном. Отражает: расхождения данных.
12. **Карточка клиента.** Устройства и история ремонтов, у мастера телефон клиента скрыт маской. Отражает: доступ на уровне поля.

Склад

13. **Склад запчастей.** Остатки в реальном времени, резервы под заказы, «последний аккумулятор уже зарезервировал мастер Иванов». Отражает: гонку за общий ресурс.
14. **Заказ запчастей поставщику.** Статусы заказа поставщику и ожидаемый срок в карточках зависимых ремонтов. Отражает: зависимость одного процесса от другого.

Фоновые операции

15. **Импорт клиентской базы.** Загрузка Excel, проверка на сервере, ошибки по строкам, фоновый импорт с прогрессом, итог «добавлено 4 210, пропущено 37, дублей 112». Отражает: длительную операцию с частичным успехом.
16. **Отчёты и выгрузки.** Очередь, прогресс, готовые файлы, ошибка с повтором, предупреждение о лимите до запуска. Отражает: фоновые задачи.
17. **SMS-рассылка.** Прогресс отправки и статус доставки по каждому получателю. Отражает: частичный успех массовой операции.
18. **Центр уведомлений и задач.** Готовые файлы, упавшие задачи, события по заказам. Отражает: единое место для всего, что сервер делает в фоне.

Доступ и служебные экраны

19. **Сотрудники и роли.** Права ролей и предпросмотр интерфейса глазами мастера. Отражает: доступ.
20. **Публичная страница статуса ремонта.** Клиент открывает её по ссылке из SMS и видит статус понятными словами, срок и согласование сметы. Страница адаптирована под телефон. Отражает: серверную логику для внешнего пользователя без входа.
21. **Технические работы.** Время окончания работ и что будет с несохранёнными данными. Отражает: недоступность сервера.
22. **Страницы ошибок (4 варианта).** Нет доступа, заказ перенесён в архив, страница не найдена, сбой сервера: у каждой своя причина и своё действие. Отражает: разные отказы сервера описаны разными словами.

Окна, компоненты, состояния

- **Модальные окна (7):**
 - конфликт правок со сравнением значений;
 - сессия истекла, черновик сохранён;
 - запчасть уже зарезервирована;
 - объединение дублей клиента;
 - подтверждение необратимого действия с последствиями («у клиента 12 заказов»);
 - отмена смены статуса;
 - повтор неудавшегося платежа.
- **Кнопка с пятью состояниями:** обычная, выполняется, заблокирована с причиной, успех, ошибка.
- **Поле с серверной проверкой:** проверяю, свободно, занято, сервер не ответил.
- **Индикаторы:** соединение (в сети, переподключение, нет сети), свежесть данных, «сейчас редактирует», прогресс фоновой задачи в шапке, статус доставки SMS.
- **Бейджи восьми статусов заказа:** цвет всегда вместе с текстом, без опоры на один цвет.
- **Уведомления:** успех, ошибка с повтором, фоновая задача готова, отменяемое действие («Статус изменён. Отменить»).
- **Недоступные действия в четырёх вариантах:** скрыто, заблокировано с причиной, только чтение, маска.
- **Скелетоны и пустые состояния** с предложением действия.
- **Кадры состояний (около 45)** для ключевых экранов 2, 5, 6, 7, 10, 13, 15 и 16.

Артефакты метода и исследование

- **Карта соответствия** «серверное состояние → экранное решение → текст пользователю»: 60-80 строк по шести аспектам.
- **Диаграммы состояний (5)**: заказ, фоновая задача, платёж, сессия, SMS.
- **Каталог паттернов отображения**: около 20 паттернов с правилами выбора.
- **Рабочий прототип в коде**. У ведущего теста есть панель, через которую он в нужный момент сценария включает задержку, обрыв сети, конфликт правок или отказ сервера.
- **Тест**: 12-20 участников, 6 сценариев с серверными событиями, два варианта интерфейса. Наивный вариант показывает «Ошибка 409», спроектированный построен по карте соответствия. Метрики: доля верно понятых ситуаций, верное следующее действие, время, SUS.

Что такое BI-система самообслуживания

BI-система самообслуживания (self-service BI) позволяет сотруднику без знания SQL и программирования самому получить ответ из данных: загрузить данные, построить график, собрать дашборд и поделиться им.

Как было в классической BI

Директор сети кофеен хочет узнать, в какой точке упала выручка в августе. Путь выглядит так:

1. Пишет заявку в IT-отдел или аналитику.
2. Аналитик пишет SQL-запрос к базе.
3. Разработчик отчётов делает отчёт.
4. Через 3-5 дней директор получает PDF.
5. Появляется уточняющий вопрос («а по дням недели?»), и всё начинается заново.

Между вопросом и ответом стоят люди, и каждый шаг занимает дни.

Как в BI-системе самообслуживания

Тот же вопрос решает сам управляющий, за 10-15 минут:

1. Подключает таблицу продаж: выгрузку из кассовой системы в Excel или готовое подключение к базе, которое заранее настроил IT.
2. Перетаскивает поле «Выручка» в область значений, а поле «Точка» на ось. Система сама строит столбчатую диаграмму.
3. Добавляет фильтр «Месяц = август» и видит, что просела точка на Ленинском.
4. Перетаскивает «День недели» и видит, что провал приходится на выходные.
5. Сохраняет график на дашборд и отправляет ссылку директору.

Уточняющий вопрос он решает сам, в тот же момент, без заявки.

Примеры систем

- **Yandex DataLens**: самый распространённый российский вариант, есть бесплатный тариф. Главный аналог для сравнения.
- **Power BI** и **Tableau**: мировые эталоны, в 2022 году ушли с российского рынка.
- **Looker Studio** от Google.
- **Metabase**: открытый код, есть конструктор «задай вопрос» без SQL.

Что делает систему «самообслуживания»

- Построение перетаскиванием полей, без кода.
- Автоматический подбор типа графика под выбранные данные.
- Визуальное связывание таблиц, без написания JOIN.
- Вычисляемые поля через простые формулы в духе Excel.
- Шаблоны дашбордов.

При этом IT никуда не исчезает: подключает источники, настраивает доступ к данным и следит за их качеством. Пользователь работает с аналитикой внутри этих рамок.

Почему это задача для дизайна

Самообслуживание постоянно ломается на человеке без аналитической подготовки, и каждая такая поломка становится задачей для интерфейса:

- **Агрегация:** система посчитала сумму там, где нужно среднее, например по цене чашки кофе, и человек этого не заметил.
- **Связи таблиц:** при связывании двух таблиц строки задвоились, и выручка выросла вдвое.
- **Тип графика:** выбран график, который искажает данные, например круговая диаграмма на 15 сегментов.
- **Термины:** «мера», «измерение», «гранулярность» ничего не говорят непрофессионалу.

Работа сводится к тому, чтобы интерфейс предотвращал такие ошибки и объяснял их понятным человеку языком. Именно это измеряется в тесте.

Пользователь без аналитической подготовки

Пользователь **без аналитической подготовки** работает с данными по своей должности, но нигде не учился их анализировать. Свою предметную область он знает хорошо. Управляющий кофейней понимает, почему в дождь падает выручка, но не знает, что такое «мера» или «связь таблиц».

Кто это

- управляющий точкой продаж;
- менеджер по продажам;
- HR-специалист, который считает текучесть кадров;
- маркетолог в небольшой компании;
- руководитель отдела, которому нужен отчёт для директора.

Что умеет и чего не умеет

Умеет

- работать в Excel на базовом уровне: фильтр, сортировка, сумма, простой график;
- читать готовые отчёты и дашборды;
- формулировать вопрос к данным словами своей области («почему упали продажи в августе»).

Не умеет

- писать SQL и понимать, как устроена база данных;
- мыслить моделью данных: меры и измерения, связи таблиц, гранулярность («одна строка = один чек или один товар в чеке»);
- различать статистические понятия: среднее и медиана, выбросы, корреляция и причинность;
- выбирать график под задачу, замечать искажения (ось не с нуля, круговая диаграмма на 15 долей);
- замечать проблемы качества данных: дубли, пропуски, разные форматы дат.

Как определить в ВКР

Понятие нужно задать через проверяемые критерии, иначе оппонент спросит, как отбирались участники теста. Шкала уровней:

- **Уровень 0.** Только читает готовые отчёты.
- **Уровень 1.** Excel на базовом уровне: фильтры, суммы, простые графики.
- **Уровень 2.** Сводные таблицы, ВПР, формулы с условиями.
- **Уровень 3.** SQL, BI-системы, статистика. Это уже аналитик.

Целевая группа работы: уровни 1 и 2. Уровень 0 скорее всего не станет строить отчёты сам, а уровню 3 упрощения не нужны.

Отбор участников: короткая анкета перед тестом

- есть ли профильное образование в анализе данных или статистике;
- писал ли когда-нибудь SQL;

- пользуется ли сводными таблицами;
- работал ли в BI-системах.

Для научной опоры есть термины «грамотность в работе с данными» (data literacy) и «грамотность в визуализации» (visualization literacy). Под второй существуют готовые тесты: VLAT (Lee, Kim, Kwon, 2017) и сокращённый Mini-VLAT на 12 вопросов (Pandey, Ottley, 2023). Mini-VLAT можно дать участникам до теста и объективно зафиксировать их уровень.

Ловушка с однокурсниками

Студенты-дизайнеры формально подходят: у них нет аналитической подготовки. Но предметной области они не знают, и мотивации управляющего у них нет. В тесте это решается легендой задачи («ты управляющий кофейней, найди причину падения выручки»). В ограничениях исследования эту особенность выборки нужно честно указать.

Как отображается серверная логика: примеры

Любое действие на экране имеет серверную сторону: сколько оно длится, чем может закончиться, что при этом происходит с данными. Интерфейс должен показать человеку каждый из этих исходов. Фуллстек-разработчик знает эти исходы, потому что сам пишет сервер.

Пример 1. Выгрузка в Excel

Как рисуют обычно. Кнопка «Выгрузить», клик, файл скачался. Один экран, одно состояние.

Что происходит на сервере

- в выборке 80 000 строк, файл собирается 40 секунд;
- прокси обрывает запрос через 60 секунд, и под нагрузкой файл не успевает собраться;
- человек не видит реакции и жмёт второй раз, сервер собирает два файла;
- человек закрывает вкладку, и работа пропадает.

Что проектируется на сервере

- выгрузка становится фоновой задачей со статусом и прогрессом;
- повторный запрос с теми же параметрами возвращает уже запущенную задачу;
- у выгрузки есть лимит строк.

Что проектируется на экране

1. Кнопка сразу переходит в состояние «Готовим файл: 12 000 из 80 000 строк» и повторно не нажимается.
2. Со страницы можно уйти, прогресс виден в шапке.
3. Когда файл готов, приходит уведомление со ссылкой.
4. При ошибке человек видит понятную причину и кнопку «Повторить».
5. Если строк больше лимита, предупреждение появляется до запуска: «В выгрузке больше 100 000 строк, выберите период короче».

Было одно состояние, стало пять: запуск, выполнение, готово, ошибка, превышен лимит.

Пример 2. Два человека правят одну запись

Как рисуют обычно. Форма, кнопка «Сохранить», сообщение «Сохранено».

Что происходит на сервере. Два оператора открыли одну заявку. Первый сменил дату визита и сохранил. Второй через минуту сохранил свою версию и молча затёр правку первого. Никто этого не заметил.

Решение

- **Сервер:** у записи есть номер версии. При сохранении устаревшей версии сервер отказывает и возвращает, что именно изменилось и кем.
- **Экран при открытии:** «Эту заявку сейчас редактирует Иванова». Событие приходит в реальном времени.
- **Экран при конфликте:** вместо «Ошибка 409» человек видит «Пока вы редактировали, Иванова изменила дату визита: было 12.09, стало 14.09» и выбирает «Оставить её значение» или «Заменить своим».

Пример 3. Сортировка в большом списке

Как рисуют обычно. Стрелки сортировки на каждой колонке таблицы.

Что происходит на сервере. В таблице 2 миллиона строк. Сортировка по колонке без индекса занимает 15 секунд.

Решение. Сортировка доступна только на колонках с индексом, у остальных стрелок нет. По умолчанию стоит фильтр «за сегодня». Набор элементов на экране прямо определяется устройством базы.

Карта соответствия: фрагмент

Центральный результат главы 3 варианта 2. Каждая строка устроена как «серверное состояние → экранное решение → текст пользователю». Фрагмент для выгрузки:

задача поставлена -> кнопка в состоянии загрузки -> «Готовим файл»

выполняется -> прогресс в шапке -> «12 000 из 80 000 строк»

готово -> уведомление со ссылкой -> «Файл готов, скачать»

ошибка -> сообщение с повтором -> «Не удалось собрать файл. Повторить»

превышен лимит -> предупреждение до запуска -> «Выберите период короче»

По карте рисуются макеты, собирается дизайн-система и прототип.

Где здесь фуллстек

1. **Знание исходов.** Дизайнер без доступа к серверу рисует удачный сценарий: он не знает, что сохранение может вернуть конфликт. Фуллстек-разработчик видит все ответы сервера, потому что сам их пишет.
2. **Реальные числа.** Решение зависит от того, сколько длится операция и сколько в таблице строк. Операции на 0,3 секунды спиннер не нужен, операции на 40 секунд нужна фоновая задача.
3. **Проектирование контракта.** Экран объяснит причину отказа, только если сервер её вернул. Одновременно решается, что отдаёт сервер («что изменилось и кем») и как это показать. Дизайнер без фуллстек-навыков получает от API голый код 409, и в интерфейсе появляется «Ошибка».
4. **Прототип с настоящим поведением.** В Figma нельзя честно показать конфликт двух пользователей или прогресс фоновой задачи. Рабочий прототип в коде это позволяет, и тест проходит на нём.

Как это проверяется

Участнику дают сценарий: «вы правите заявку и нажимаете Сохранить», а на экране возникает конфликт. Затем спрашивают: «Что сейчас произошло с вашими данными и что вы будете делать?» Сравнивается доля правильных ответов на двух вариантах. Исходный вариант показывает «Ошибка 409» или молча перезаписывает данные, спроектированный объясняет конфликт и даёт выбор.